Gabriella Tamayo CSC 647 - Spring 2026

Project 5: Eridianese Translator

1. Abstract

This project presents a GPU-accelerated frequency-to-text translation system inspired by the musical communication method in Andy Weir's *Project Hail Mary*. The system processes WAV audio files containing sinusoidal tones mapped to vocabulary words through frequency lookup. The core computation performs Fast Fourier Transforms (FFT) on audio chunks to identify dominant frequencies and reconstruct sentences. A serial C baseline was compared against a CUDA implementation leveraging NVIDIA's cuFFT library and custom kernels with shared memory for parallel amplitude calculation and reduction. Performance was measured across test files of increasing size, with per-operation timing for FFT, amplitude, and maximum-finding stages. Results demonstrate significant FFT speedup through cuFFT, while smaller O(N) operations show limited GPU advantage due to kernel launch overhead. This work illustrates practical GPU computing techniques such as shared memory, parallel reduction, and library-based FFT acceleration which is applied to audio signal processing, while highlighting the importance of matching problem size to hardware architecture.

2. Background

2.1. The Story and Musical Communication

Andy Weir's *Project Hail Mary* presents a compelling scenario in which the protagonist, Dr. Ryland Grace, must communicate with an alien species using a fundamentally different sensory modality. The Eridians, native to a planet with a dense atmosphere, perceive the world primarily through sound rather than sight. Their language consists of musical chords, with each chord representing a distinct word or concept. Communication between human and Eridian becomes possible through the universal language of frequencies by mapping specific tones to specific meanings.

In the novel, this translation occurs through trial and error, at first, before relying on fast fourier transforms (FFT) that Grace runs in real-time on a laptop to translate the musical language into English words. With every new musical chord, he determines an English-worded counterpart and adds it to a spreadsheet so future communication progressively becomes easier. While this is a work of fiction, the science behind this communication is sound. One could easily recreate this translator that utilizes FFTs, but could this workflow be parallelized for further optimization?

2.2. The Fourier Transform and Frequency Detection

The fundamental mathematical tool enabling this translation is the Fourier Transform. In essence, any complex sound wave can be decomposed into a sum of simple sine waves at different frequencies, each with its own amplitude. The Fourier Transform performs this decomposition, converting a signal from the time domain (amplitude over time) to the frequency domain (amplitude at each frequency).

For discrete digital audio, we use the Discrete Fourier Transform (DFT). However, computing the DFT directly requires $O(N^2)$ operations. For our 8192-sample chunks, this would mean roughly 67 million operations per chunk. The Fast Fourier Transform (FFT), developed by Cooley and Tukey in 1965, reduces this to $O(N \log N)$ operations through a divide-and-conquer strategy. The algorithm recursively splits the input into even and odd-indexed elements, computes their FFTs separately, then combines results using "butterfly" operations involving complex number multiplication by twiddle factors. For 8192 samples, the FFT requires approximately 106,000 operations rather than 67 million which is a 630-fold reduction.

The output of an FFT on real-valued audio data is a set of complex numbers representing frequency bins. Each bin corresponds to a frequency range of width RATE/FFT_SIZE (approximately 5.38 Hz in our configuration). The magnitude of each complex number indicates the strength of that frequency component. By identifying the bin with the maximum magnitude, we determine the dominant frequency present in the audio chunk. This detected frequency is then compared against a vocabulary table to find the closest matching word within a specified tolerance.

3. Serial Algorithm

3.1. Original Python Implementation

The starting point for this project was an open-source Python script by Sam Knight designed for real-time chord-to-speech translation using a microphone. The original code captured live audio through PyAudio, performed FFT analysis using NumPy's optimized implementation, detected the dominant frequency, matched it to a vocabulary entry from a CSV file, and optionally passed the assembled sentence through Claude for grammar correction before converting to speech via ElevenLabs text-to-speech.

Several adaptations were necessary to transform this interactive, cloud-dependent program into a batch-processing system suitable for GPU acceleration benchmarking. The most significant change was replacing live microphone input with WAV file processing, which eliminated the dependency on PyAudio, removed the need for physical audio hardware on the GPU cluster, and enabled reproducible testing with known input files. The Claude and ElevenLabs integrations were removed as unnecessary for a computational performance study. The matplotlib spectrum visualization was similarly stripped away, leaving only the core signal processing pipeline.

Parameter tuning proved essential during development. The sample rate was increased from the original 4410 Hz to 44100 Hz to support higher frequencies in the extended vocabulary (words up to 5274 Hz require a sample rate of at least double that per the Nyquist theorem). The chunk size was increased from 1024 to 8192 samples, providing frequency resolution of 5.38 Hz rather than the original 43 Hz. The frequency matching tolerance was adjusted to 10 Hz to accommodate the residual bin alignment error while avoiding word collisions. A WAV file generation script was created to

produce test files containing sequences of known frequencies with controlled durations and inter-word silence gaps.

3.2. Conversion to C

The Python implementation served as a functional reference, but GPU programming with CUDA requires C/C++. The translation to C involved rewriting the FFT algorithm, audio processing pipeline, and WAV file parsing from scratch. The original NumPy FFT calls were replaced with a hand-written recursive Cooley-Tukey implementation using separate real and imaginary float arrays rather than complex number structures. The WAV file reader was implemented from first principles, parsing the RIFF header format to extract sample rate, channel count, bit depth, and raw audio data.

Several Python features were intentionally omitted from the C translation. Python's dynamic memory management was replaced with explicit malloc/free calls with fixed maximum array sizes defined through preprocessor constants. The threading-based phrase timer was simplified to an inline time check between chunk iterations. Python's string manipulation and list operations were replaced with C-style fixed-size character arrays. These simplifications produced a clean, self-contained C program suitable for compilation with both standard GCC (for CPU benchmarking) and NVCC (for GPU CUDA compilation).

4. Parallelization Strategy

4.1. GPU Kernel Architecture

The GPU implementation divides the per-chunk computation into three distinct parallel operations. First, the FFT is performed using NVIDIA's cuFFT library through its `cufftExecR2C` function, which executes a highly optimized real-to-complex transform entirely on the GPU. cuFFT handles all internal parallelism, memory management, and algorithmic optimizations, providing the fastest possible FFT implementation for NVIDIA hardware. A cuFFT plan is created once during initialization and reused across all chunks to amortize setup overhead.

Second, a custom kernel computes amplitude magnitudes from the cuFFT output. cuFFT's real-to-complex transform produces interleaved complex data—alternating real and imaginary values in a single array. Each GPU thread reads one complex pair, computes $\text{sqrt}(\text{real}^2 + \text{imag}^2)$, and writes the result to an amplitude array. This kernel employs a straightforward one-thread-per-element mapping over the 4096 positive frequency bins with 256 threads per block, requiring no inter-thread communication.

Third, the maximum amplitude is found using a shared memory reduction kernel. The 4096 amplitude values are distributed across 16 thread blocks of 256 threads each. Within each block, threads load their values into shared memory and perform a tree-based parallel reduction over $\log_2(256) = 8$ steps, with `__syncthreads()` barriers ensuring correct ordering. Each block produces a single local maximum, and the resulting 16 values are transferred to the CPU to trivially determine the global

maximum. This approach avoids the performance penalty of atomic operations on global memory while demonstrating the shared memory optimization technique.

4.2. Challenges with Custom GPU FFT Implementation

An initial attempt was made to implement the FFT directly as a custom CUDA kernel using shared memory for the butterfly operations. The iterative approach launched one kernel per FFT stage, with each thread handling one butterfly pair. Despite careful attention to indexing and twiddle factor computation, this implementation produced incorrect frequency outputs. Even after prompting the Large Language Model (LLM) DeepSeek to handle this problem and give it freedom to solve the issue by rewriting the script, correct results could not be yielded. The challenge lay in the complex data access patterns required by the Cooley-Tukey algorithm. Each FFT stage pairs elements separated by a stride that doubles each iteration, and the bit-reversal permutation required prior to the first stage introduces additional complexity. Debugging incorrect FFT results proved difficult because errors in any single stage propagate through all subsequent stages. The decision to adopt cuFFT, NVIDIA's thoroughly tested and optimized library, ensured correctness while providing superior performance compared to CPU performance.

5. Performance Analysis

5.1. Timing Results

Performance measurements were collected across ten WAV files containing 1 to 20 words in alternating frequency pairs, with total samples ranging from approximately 120,000 to 1,140,000. Per-operation timing was measured using CUDA events for GPU kernels and clock() for CPU operations, isolating the FFT, amplitude calculation, and maximum-finding stages.

The FFT computation demonstrated the most significant GPU advantage. For the largest test file (138 chunks), the CPU's recursive FFT implementation required 0.220900 seconds while cuFFT completed in 0.001582 seconds, demonstrating a 139-fold speedup. This dramatic improvement reflects cuFFT's use of parallel butterfly operations across thousands of GPU cores, optimized memory access patterns, and hardware-specific tuning that far exceeds the capabilities of a single-threaded recursive C implementation. As problem size increased, the GPU advantage remained consistently strong, with per-chunk FFT time staying nearly constant on the GPU while growing linearly on the CPU.

Amplitude calculation showed more modest GPU improvement. The GPU kernel achieved approximately 5-fold speedup over the CPU loop for the largest test file, but the absolute times were small (microseconds per chunk) on both platforms. The GPU advantage is tempered by kernel launch overhead and proves to be a significant fraction of the total time for such a small operation.

Maximum-finding produced the most instructive result: the CPU was faster than the GPU. The CPU requires only a single loop over 4096 elements with a running maximum, executing in approximately 32 microseconds per chunk. To be clear, each element is one frequency bin from the

FFT output. The GPU kernel, despite using shared memory reduction, incurs kernel launch latency, and thread synchronization overhead. For 4096 elements, these overheads exceed the actual computation time. This outcome effectively demonstrates that GPU acceleration is beneficial only when computational work sufficiently outweighs scheduling and communication costs. If the WAV files were significantly longer, parallelization may then prove useful.

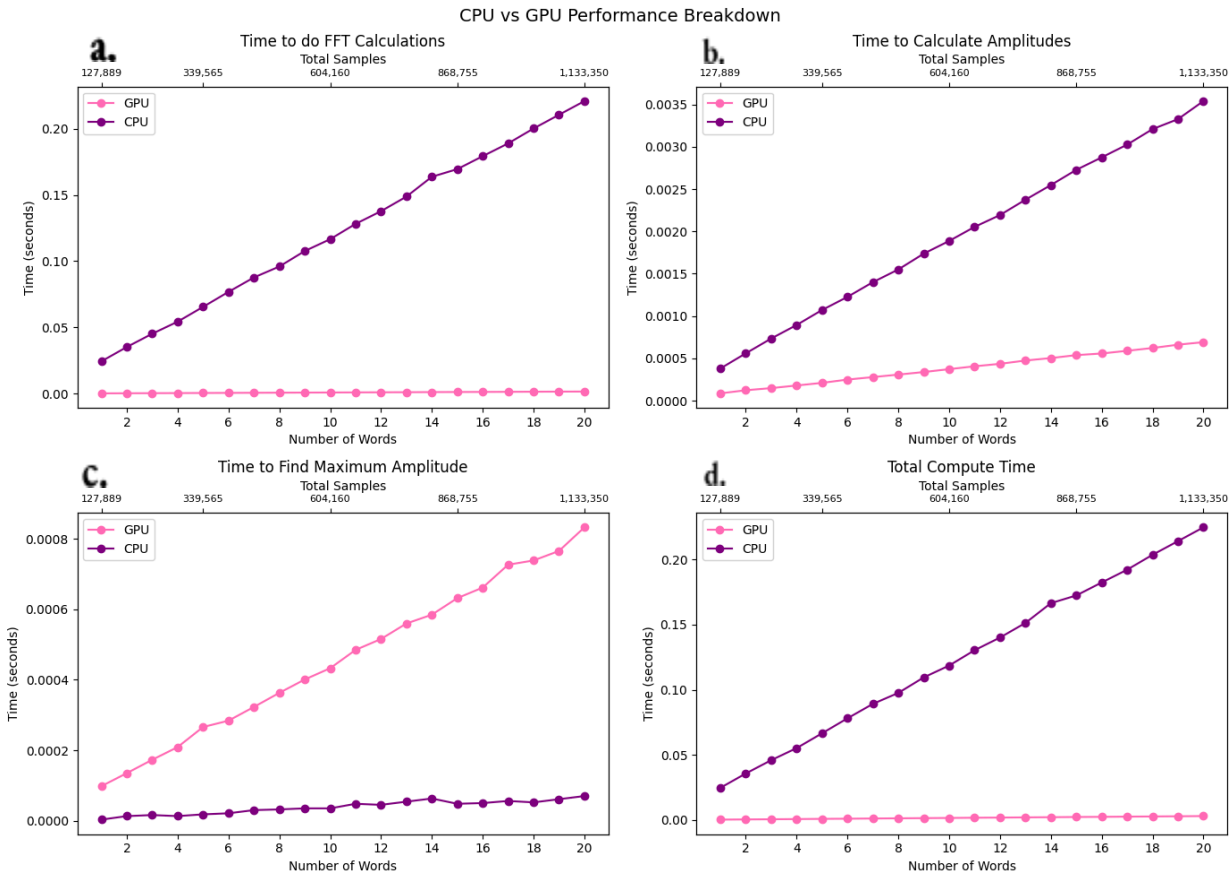

Figure 1: Performance comparison between the GPU (pink) and CPU (purple) for the execution time of the (a) FFT Calculation, (b) Amplitude Calculation, (c) Maximum Amplitude Finding, and (d) Total Computation which is a sum of the former 3 times.

5.2. Benefits and Drawbacks

The primary benefit of GPU acceleration for this application lies in the FFT computation, which dominates the processing pipeline. The 139-fold FFT speedup translates directly to faster overall processing, particularly for longer audio files with many chunks. For real-time applications requiring low latency, this is an obvious improvement yielding faster calculations. The use of cuFFT also provides access to ongoing NVIDIA optimizations without requiring changes to application code.

The principal drawback is the limited parallelizability of the remaining pipeline stages. Amplitude calculation and maximum-finding are O(N) operations on relatively small N, making them poor candidates for GPU acceleration. The GPU memory allocation, data transfer, and kernel launch

overhead for each chunk further erode potential gains. Only the kernel function was timed, but there is obvious overhead from all other components. Additionally, the dependence on NVIDIA hardware and the CUDA ecosystem limits portability compared to the CPU-only implementation.

6. Conclusions

This project successfully demonstrated the application of GPU computing techniques to a frequency-based audio translation system. Starting from a Python prototype, a serial C implementation was developed and subsequently parallelized using CUDA with cuFFT for FFT acceleration and custom kernels employing shared memory for amplitude calculation and parallel reduction. The system correctly translates frequency-encoded WAV files into their corresponding sentences, validating both the computational approach and the GPU implementation.

Performance analysis revealed that GPU acceleration provides substantial benefits for the computationally intensive FFT stage, achieving a 139-fold speedup through cuFFT. However, the smaller O(N) operations showed diminishing returns from parallelization, with the maximum-finding stage actually performing worse on GPU due to kernel launch overhead. These results highlight the critical importance of how some calculations do not always achieve better performance when parallelized.

LLM Usage Disclosure

The use of the Large Language Model "DeepSeek" was used for both troubleshooting code and aid in writing this report. The entire conversation can be found here:
https://chat.deepseek.com/share/ufrmtp9bc48p45wiaa

Prompts for the report:

1. "This project was based off of the story: project hail mary. I need to write a 5-10 page report on the entirety of the project now that it's completed. Construct an abstract for the beginning of this report that is appropriate for the graduate student level."
    - After reading the generated abstract, I further prompted it: "Condense this without sharing too many details and rewrite to be more concise." After reading the concise version, I edited a few small details and double checked that the information given was indeed what this project was trying to demonstrate.
2. "Here is an image of a general outline for the report. Write the report based on this outline that is appropriate for the graduate student level. You can optionally create sections and subsections for clarity such as that seen in the outline. The report should be 5 pages. Output with bolded section headers that are numbered."
    - Please refer to the image under "Additional Information" to see what was uploaded to the LLM.
    - Since this was prompted in the same conversation as the code troubleshooting, the LLM was able to formulate a mostly correct report with relevant sections and subsections which broke up the report clearly and had a good flow. There were problems with the values under "Section 5.1 Timing Results" that were generated as expected since I never uploaded the data, only the scripts as a final check. I referenced my dataset and corrected the values.
    - There were also a few misconceptions in the beginning with the original python script. The LLM did not understand that this was the work of someone else, so I edited what was relevant to give credit and explain what the original code was responsible for.
    - The entire report was read and slightly edited as well as cross checked by verifying certain facts/statements with Google's search engine. I further prompted DeepSeek to clarify certain variables and parameters to be sure it was referenced correctly.

Additional Information

1. **Abstract**
    1.1. DONE
2. **Background**
    2.1. The story / How it's done in the book
    2.2. What is FFT?
        - Explain chords to notes
3. **Serial Algorithm**
    3.1. Original code found on github in python
        - What does the code do?
        - What differs from the story?
        - What needs to be changed in this code?
            - Input .wav files
        - Troubleshooting
            - Changing parameters
            - Creating many .wav files
    3.2. Converting to C
        - What do we not use from the original code?
4. **Parallelization Strategy**
    4.1. Three functions
        - cuFFT, kernel for calculating amplitudes, kernel for finding maximum
        - What does each do?
        - How is each optimized?
    4.2. Problems with parallelizing the original FFT script
        - What was tried?
5. **Performance**
    5.1. Talk about four speedup plots
        - Speedup for total compute time, amplitude calculation, and cuFFT was as expected with GPU doing much better than CPU. Finding maximum speedup showed the GPU doing worse.
    5.2. Benefits and drawbacks
6. **Conclusions**
    6.1. Summary

Image 1: Screenshot of the PNG uploaded to DeepSeek with Prompt 2.